

Parallel Programming Assignment Report Template

Students Information

CS3

كيرلس أشرف اديب جوده

كيرلس هاني حلمي ملك

فادي عماد ثابت

كريم محمد سيد

كيرلس جاب الله فتحي

مصطفى سيد إسماعيل

Problem 1

Problem Description

A basic program to print a greeting message along with the thread ID. The objective is to provide a baseline for serial execution.

Parallelization Approach

None (Serial execution).

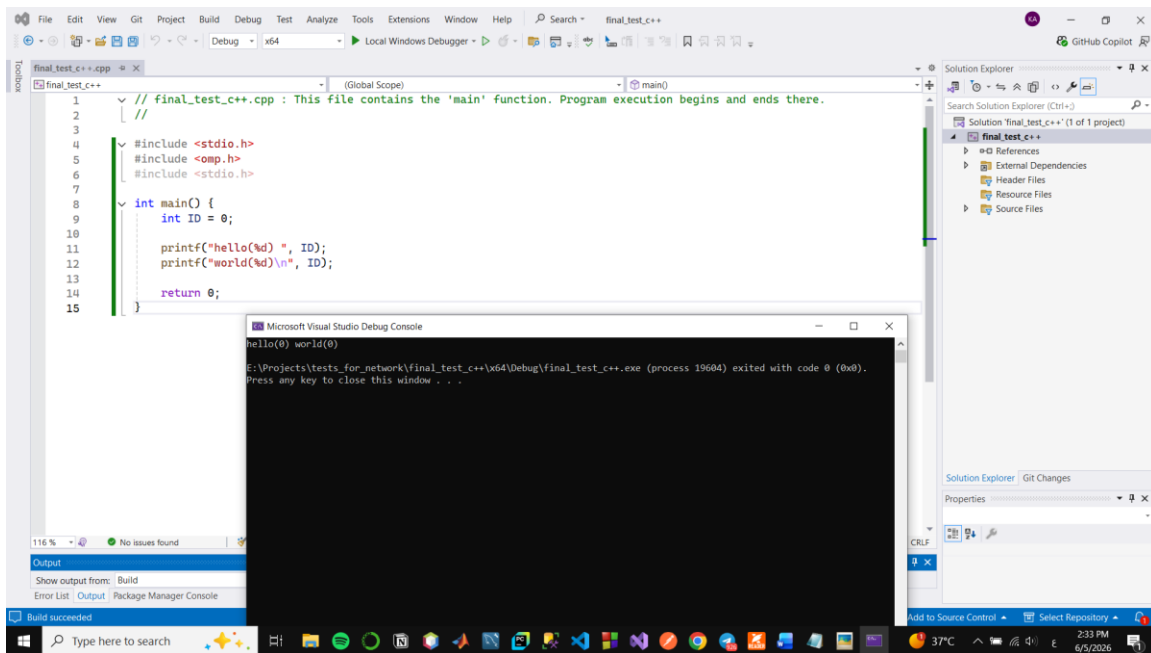
Solution Explanation

The program initializes an `ID` variable to 0 and uses `printf` to print the message. All instructions are executed sequentially by the single Main Thread.

SOURCE CODE

```
1 Hello World (Serial Version)
2 #include <stdio.h>
3
4 int main() {
5     int ID = 0;
6
7     printf("hello(%d) ", ID);
8     printf("world(%d)\n", ID);
9
10    return 0;
11 }
```

Execution Results



The expected output is printing `hello(0) world(0)` exactly once on the screen.

Conclusion

This program serves as a foundational reference to ensure the environment is set up correctly before applying parallel processing concepts.

Problem 2

Problem Information

Problem Title Hello World (Parallel Version)

Lecture Number 3

Problem Description

Converting the Hello World program to run in parallel, where each thread prints the greeting along with its unique ID.

Parallelization Approach

Using the ``#pragma omp parallel`` directive to create a parallel region and ``omp_get_thread_num()`` to fetch each thread's ID.

Solution Explanation

Upon reaching the parallel region, threads are spawned based on the system's default count. Each thread queries its ID and prints it alongside hello world.

Source Code



```
1 (Parallel Version)
2 #include <stdio.h>
3 #include <omp.h>
4
5
6 int main() {
7
8     #pragma omp parallel
9         {
10             int ID = omp_get_thread_num();
11
12             printf("hello(%d) ", ID);
13             printf("world(%d)\n", ID);
14         }
15
16     return 0;
17 }
18
```

Execution Results

The message is printed multiple times (based on available threads), with different ID numbers (e.g., 0 to 3), in an unpredictable order due to race conditions on the standard output.

```
1 // final_test_c++.cpp : This file contains the 'main' function. Program execution begins and ends there.
2 //
3
4 #include <stdio.h>
5 #include <omp.h>
6 #include <stdio.h>
7
8 #include <stdio.h>
9 #include <omp.h>
10
11 int main() {
12     #pragma omp parallel
13     {
14         int ID = omp_get_thread_num();
15
16         printf("hello(%d) ", ID);
17         printf("world(%d)\n", ID);
18     }
19
20     return 0;
21 }
```

Microsoft Visual Studio Debug Console

```
hello(1) world(1)
hello(10) world(10)
hello(11) hello(12) world(11)
hello(5) world(5)
hello(7) world(7)
hello(11) world(11)
hello(4) world(4)
hello(0) world(0)
hello(12)
hello(2) world(2)
hello(6) world(6)
hello(3) world(3)
hello(8) world(8)
hello(15) world(15)
hello(14) world(14)
hello(9) world(9)
```

Build succeeded

Conclusion

This demonstrates how easily threads can be spawned in OpenMP and highlights that thread execution order is never guaranteed.

Problem 3

Problem Information

Problem Title Pi Calculation (Serial Version)

Lecture Number 3

Problem Description

Calculating the approximate value of the mathematical constant π using numerical integration (calculating the area of rectangles under a curve).

Parallelization Approach

None (Serial execution).

Solution Explanation

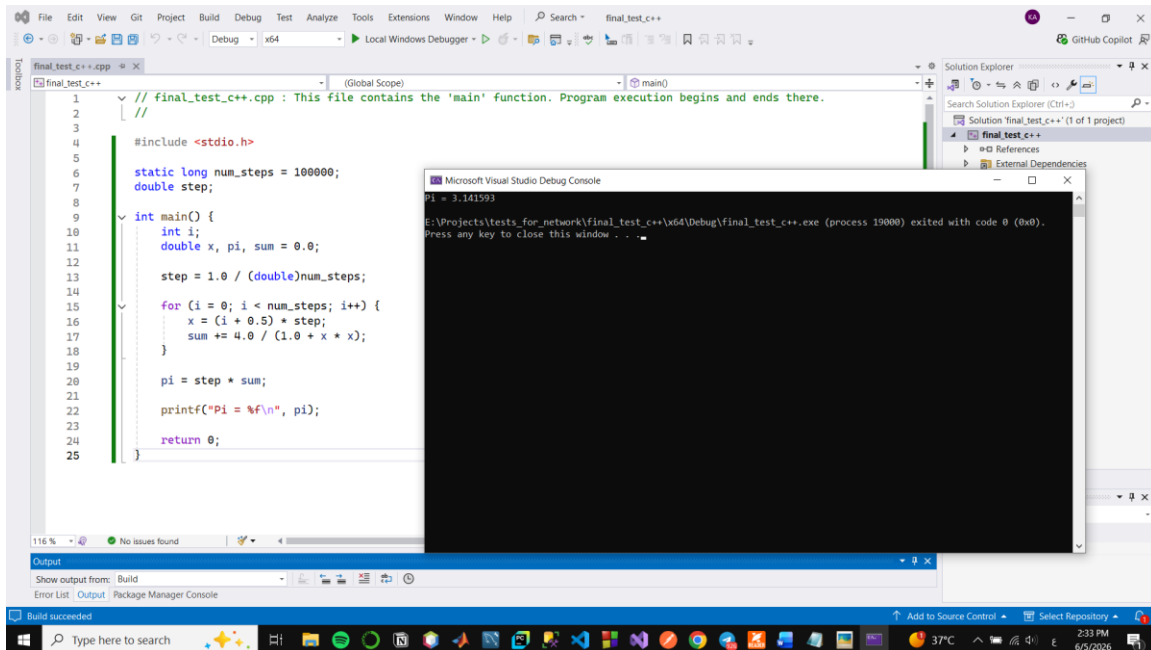
The total area is divided into 100,000 steps. A `for` loop iterates through each step to calculate the height and accumulate the `sum`. Finally, the total sum is multiplied by the step width.

Source Code

```
1  #include <stdio.h>
2
3  static long num_steps = 100000;
4  double step;
5
6  int main()
7  {
8      int i;
9      double x, pi, sum = 0.0;
10
11     step = 1.0 / (double)num_steps;
12
13     for (i = 0; i < num_steps; i++)
14     {
15         x = (i + 0.5) * step;
16         sum += 4.0 / (1.0 + x * x);
17     }
18
19     pi = step * sum;
20
21     printf("Pi = %f\n", pi);
22
23     return 0;
24 }
25
```

Execution Results

Accurately prints the value of Pi (approx. 3.14159). The execution time serves as the baseline to measure the efficiency of future parallel optimizations.



The screenshot shows the Visual Studio IDE with a C++ file named `final_test_c++.cpp` open. The code implements a sequential Monte Carlo method for calculating Pi. It defines `num_steps = 100000` and a `step` variable. The `main` function initializes `pi` and `sum` to 0.0, then enters a loop where it generates random points `x` and `y` (using `rand() / RAND_MAX`) and checks if they fall within a unit circle. The sum of points inside the circle is used to calculate `pi` as `4.0 * sum / num_steps`. The final value of `pi` is printed using `printf`. A debug console window is open, showing the output: `pi = 3.141593`. The status bar at the bottom indicates 'Build succeeded'.

```
1 // final_test_c++.cpp : This file contains the 'main' function. Program execution begins and ends there.
2
3
4 #include <stdio.h>
5
6 static long num_steps = 100000;
7 double step;
8
9 int main() {
10     int i;
11     double x, pi, sum = 0.0;
12
13     step = 1.0 / (double)num_steps;
14
15     for (i = 0; i < num_steps; i++) {
16         x = (i + 0.5) * step;
17         sum += 4.0 / (1.0 + x * x);
18     }
19
20     pi = step * sum;
21
22     printf("Pi = %f\n", pi);
23
24     return 0;
25 }
```

Microsoft Visual Studio Debug Console

```
pi = 3.141593
E:\Projects\tests_for_network\final_test_c++\x64\Debug\final_test_c++.exe (process 19000) exited with code 0 (0x0).
Press any key to close this window . . .
```

Conclusion

The algorithm is mathematically precise but becomes slow for massive inputs when executed sequentially on a single core.

Problem 4

Problem Information

Problem Title Pi Calculation (Parallel SPMD Version)

Lecture Number 3

Problem Description

Accelerating the Pi calculation by distributing loop iterations across multiple threads using the SPMD (Single Program Multiple Data) model.

Parallelization Approach

Manual work division inside `#pragma omp parallel`. Each thread starts at its unique ID, jumps by the total number of threads, and stores its local result in a `sum` array.

Solution Explanation

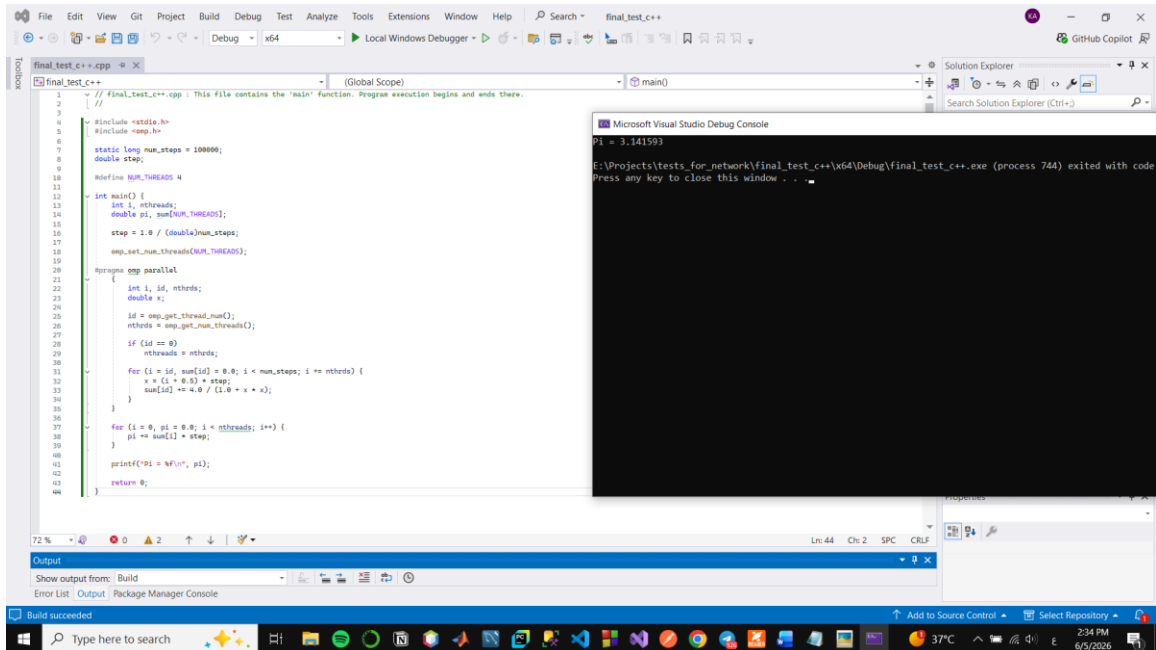
Instead of a single long loop, each thread accumulates its assigned values into a specific index in the `sum[NUM\_THREADS]` array. After the parallel region, the master thread sums the array elements for the final result.

Source Code

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  static long num_steps = 100000;
5  double step;
6
7  #define NUM_THREADS 4
8
9  int main()
10 {
11     int i, nthreads;
12     double pi, sum[NUM_THREADS];
13
14     step = 1.0 / (double)num_steps;
15
16     omp_set_num_threads(NUM_THREADS);
17
18     #pragma omp parallel
19     {
20         int i, id, nthrds;
21         double x;
22
23         id = omp_get_thread_num();
24         nthrds = omp_get_num_threads();
25
26         if (id == 0)
27             nthreads = nthrds;
28
29         for (i = id, sum[id] = 0.0; i < num_steps; i += nthrds)
30         {
31             x = (i + 0.5) * step;
32             sum[id] += 4.0 / (1.0 + x * x);
33         }
34     }
35
36     for (i = 0, pi = 0.0; i < nthreads; i++)
37     {
38         pi += sum[i] * step;
39     }
40
41     printf("Pi = %f\n", pi);
42
43     return 0;
44 }
```

Execution Results

Computes Pi correctly, but performance might be poor due to False Sharing, which occurs when different threads write to adjacent array elements located on the same cache line.



The screenshot shows the Visual Studio IDE with a C++ file named `final_test_c++.cpp` open. The code is a parallel implementation of the Monte Carlo method for calculating Pi. It uses `#pragma omp parallel` to create multiple threads. The console output shows `Pi = 3.141593`. The status bar at the bottom indicates 'Build succeeded'.

```
1 // final_test_c++.cpp : This file contains the 'main' function. Program execution begins and ends there.
2
3 #include <stdio.h>
4 #include <omp.h>
5 static long num_steps = 100000;
6 double step;
7
8 #define NUM_THREADS 4
9
10 int main() {
11     int i, nthreads;
12     double pi;
13     double x;
14     step = 1.0 / (double)num_steps;
15     omp_set_num_threads(NUM_THREADS);
16
17     #pragma omp parallel
18     {
19         int i, id, nthreads;
20         double x;
21         id = omp_get_thread_num();
22         nthreads = omp_get_num_threads();
23         if (id == 0)
24             nthreads = nthreads;
25
26         for (i = 0, sum[id] = 0.0; i < num_steps; i += nthreads) {
27             x = (i + 0.5) * step;
28             sum[id] += 4.0 / (1.0 + x * x);
29         }
30
31         for (i = 0, pi = 0.0; i < nthreads; i++) {
32             pi += sum[i] * step;
33         }
34
35         printf("Pi = %f\n", pi);
36     }
37     return 0;
38 }
```

Conclusion

While logically correct, the subpar performance highlights the importance of understanding underlying hardware and cache architecture in parallel programming.

Problem 5

Problem Information

Problem Title Pi Calculation (Array Padding Version)

Lecture Number 3

Problem Description

Solving the False Sharing problem observed in the SPMD version to improve actual execution performance.

Parallelization Approach

Modifying the data structure using Memory Padding. The array becomes a 2D array `sum[NUM_THREADS][PAD]`, where PAD represents the size of a CPU cache line.

Solution Explanation

The algorithm matches the SPMD version, but changing the array definition ensures that each thread's accumulation variable resides on a completely independent cache line, preventing hardware-level cache invalidation conflicts.

Source Code

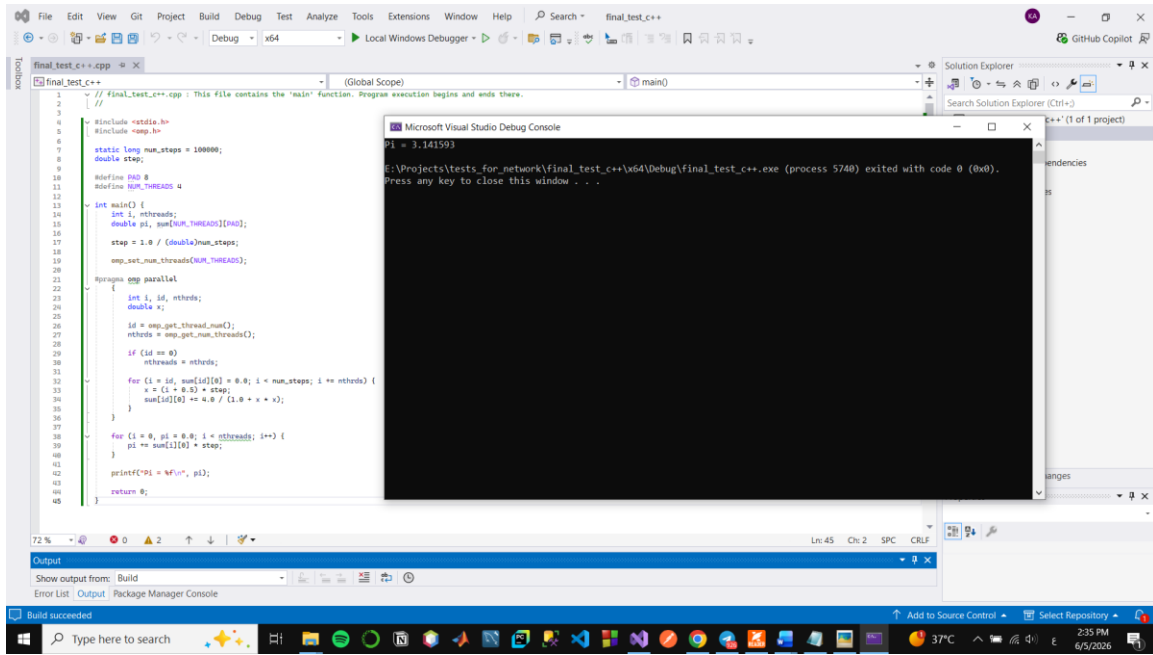
```

1  #include <stdio.h>
2  #include <omp.h>
3
4  static long num_steps = 100000;
5  double step;
6
7  #define PAD 8
8  #define NUM_THREADS 4
9
10 int main()
11 {
12     int i, nthreads;
13     double pi, sum[NUM_THREADS][PAD];
14
15     step = 1.0 / (double)num_steps;
16
17     omp_set_num_threads(NUM_THREADS);
18
19     #pragma omp parallel
20     {
21         int i, id, nthrds;
22         double x;
23
24         id = omp_get_thread_num();
25         nthrds = omp_get_num_threads();
26
27         if (id == 0)
28             nthreads = nthrds;
29
30         for (i = id, sum[id][0] = 0.0; i < num_steps; i += nthrds)
31         {
32             x = (i + 0.5) * step;
33             sum[id][0] += 4.0 / (1.0 + x * x);
34         }
35     }
36
37     for (i = 0, pi = 0.0; i < nthreads; i++)
38     {
39         pi += sum[i][0] * step;
40     }
41
42     printf("Pi = %f\n", pi);
43
44     return 0;
45 }
46

```

Execution Results

Significant reduction in execution time compared to the basic SPMD version, while maintaining accurate Pi calculation.



Conclusion

Padding is a highly effective technique for mitigating False Sharing, though it increases memory consumption and slightly complicates the code.

Problem 6

Problem Information

Problem Title Pi Calculation (Critical Section Version)

Lecture Number 3

Problem Description

Simplifying the result accumulation code, eliminating arrays and padding, while ensuring the thread-safe addition of the final sum.

Parallelization Approach

Using a local `sum` variable for each thread, then adding this local sum to the shared `pi` variable inside a protected `#pragma omp critical` region.

Solution Explanation

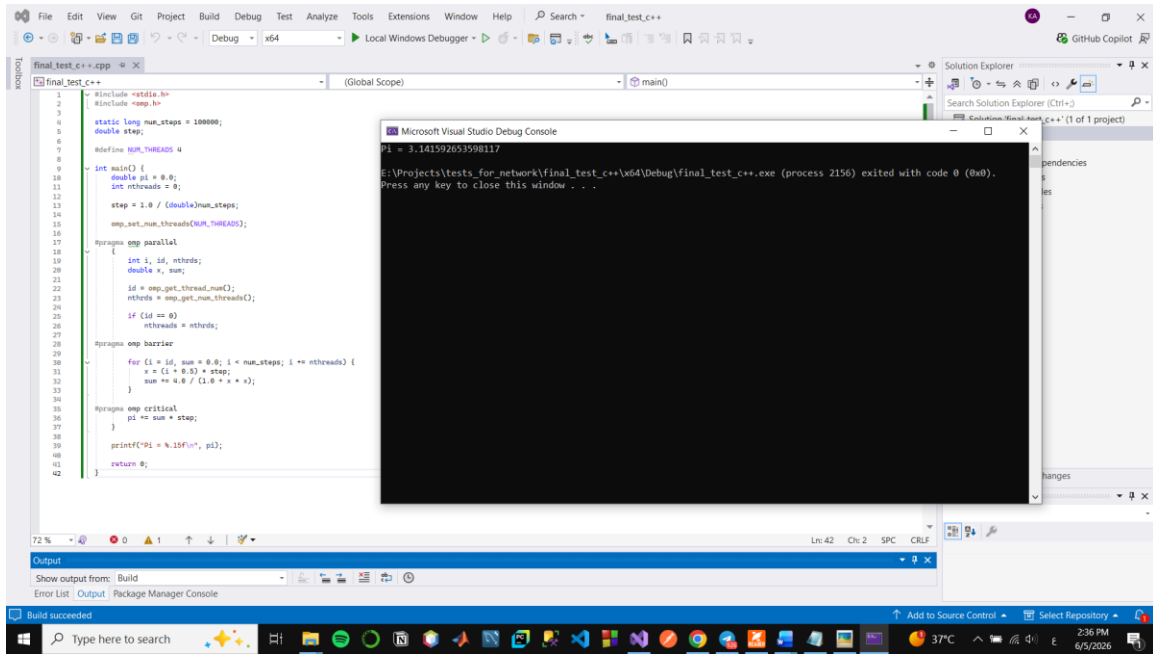
Each thread performs its calculations independently using its private variable. Once finished, threads enter the `critical` section one by one to safely update the global `pi` variable, preventing race conditions.

Source Code

```
1 #include <stdio.h>
2 #include <omp.h>
3
4 static long num_steps = 100000;
5 double step;
6
7 #define NUM_THREADS 4
8
9 int main()
10 {
11     double pi = 0.0;
12     int nthreads = 0;
13
14     step = 1.0 / (double)num_steps;
15
16     omp_set_num_threads(NUM_THREADS);
17
18 #pragma omp parallel
19 {
20     int i, id, nthrds;
21     double x, sum;
22
23     id = omp_get_thread_num();
24     nthrds = omp_get_num_threads();
25
26     if (id == 0)
27         nthreads = nthrds;
28
29 #pragma omp barrier
30
31     for (i = id, sum = 0.0; i < num_steps; i += nthrds)
32     {
33         x = (i + 0.5) * step;
34         sum += 4.0 / (1.0 + x * x);
35     }
36
37 #pragma omp critical
38     pi += sum * step;
39 }
40
41 printf("Pi = %.15f\n", pi);
42
43 return 0;
44 }
45
```

Execution Results

Accurate Pi calculation. Excellent performance, very close to optimal, because the lock (critical section) is executed only once per thread at the very end of its workload.



Conclusion

Synchronization using critical sections is safe and effective but can become a massive performance bottleneck if placed inside high-frequency loops.

Problem 7

Problem Information

Problem Title Pi Calculation (Atomic Version)

Lecture Number 3

Problem Description

Optimizing the final update of the shared `pi` variable to be faster and less resource-intensive than using a full critical section.

Parallelization Approach

Utilizing the ``#pragma omp atomic`` directive instead of a critical section to safely add local variables to the global variable.

Solution Explanation

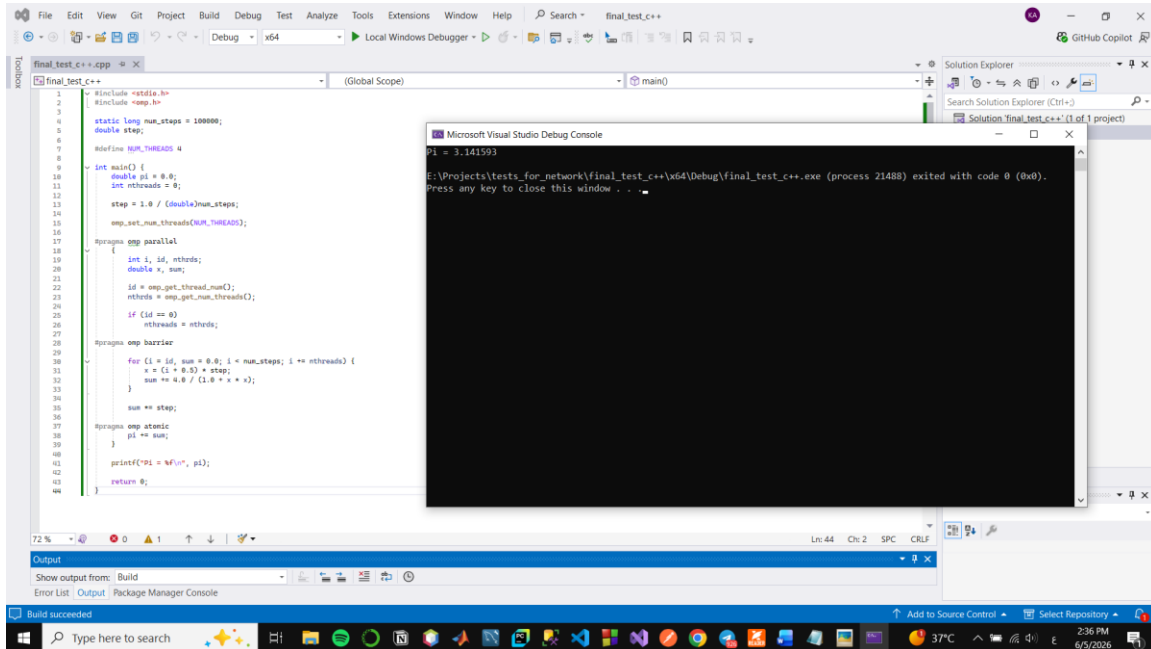
The ``atomic`` directive leverages hardware capabilities to execute a specific operation (like addition) in a single, indivisible step. This has much less overhead than software locks used by ``critical``.

Source Code


```
1 #include <stdio.h>
2 #include <omp.h>
3
4     static long num_steps = 100000;
5 double step;
6
7 #define NUM_THREADS 4
8
9 int main()
10 {
11     double pi = 0.0;
12     int nthreads = 0;
13
14     step = 1.0 / (double)num_steps;
15
16     omp_set_num_threads(NUM_THREADS);
17
18 #pragma omp parallel
19 {
20     int i, id, nthrds;
21     double x, sum;
22
23     id = omp_get_thread_num();
24     nthrds = omp_get_num_threads();
25
26     if (id == 0)
27         nthreads = nthrds;
28
29 #pragma omp barrier
30
31     for (i = id, sum = 0.0; i < num_steps; i += nthrds)
32     {
33         x = (i + 0.5) * step;
34         sum += 4.0 / (1.0 + x * x);
35     }
36
37     sum *= step;
38
39 #pragma omp atomic
40     pi += sum;
41 }
42
43 printf("Pi = %f\n", pi);
44
45 return 0;
46 }
```

Execution Results

Accurate calculations with slightly better (or on-par) performance compared to the critical section due to reduced synchronization overhead.



```
1 #include <stdio.h>
2 #include <omp.h>
3
4 static long num_steps = 100000;
5 double step;
6
7 #define NUM_THREADS 4
8
9 int main() {
10     double pi = 0.0;
11     int nthreads = 0;
12
13     step = 1.0 / (double)num_steps;
14
15     omp_set_num_threads(NUM_THREADS);
16
17     #pragma omp parallel
18     {
19         int i, id, nthreads;
20         double x, sum;
21
22         id = omp_get_thread_num();
23         nthreads = omp_get_num_threads();
24
25         if (id == 0)
26             nthreads = nthreads;
27
28         #pragma omp barrier
29
30         for (i = id, sum = 0.0; i < num_steps; i += nthreads) {
31             x = (i + 0.5) * step;
32             sum += 4.0 / (1.0 + x * x);
33         }
34
35         sum *= step;
36
37         #pragma omp atomic
38         pi += sum;
39     }
40
41     printf("pi = %f\n", pi);
42     return 0;
43 }
```

Microsoft Visual Studio Debug Console

```
pi = 3.141593
E:\Projects\tests_for_network\final_test_c++\x64\Debug\final_test_c++.exe (process 21488) exited with code 0 (0x0).
Press any key to close this window . . .
```

Conclusion

`atomic` should always be preferred over `critical` when the operation is simple enough to be natively supported by hardware atomic instructions.

Problem 8

Problem Information

Problem Title Pi Calculation (Reduction Version)

Lecture Number 3

Problem Description

Utilizing built-in OpenMP features to simplify the code and achieve maximum parallel safety without manual lock management.

Parallelization Approach

Using `#pragma omp parallel for` for automatic loop distribution, combined with the `reduction(+sum)` clause to handle the accumulative addition safely.

Solution Explanation

OpenMP automatically creates private copies of the `sum` variable for each thread, then implicitly manages the final addition of all local copies into the master variable. This eliminates the need for arrays or manual critical sections.

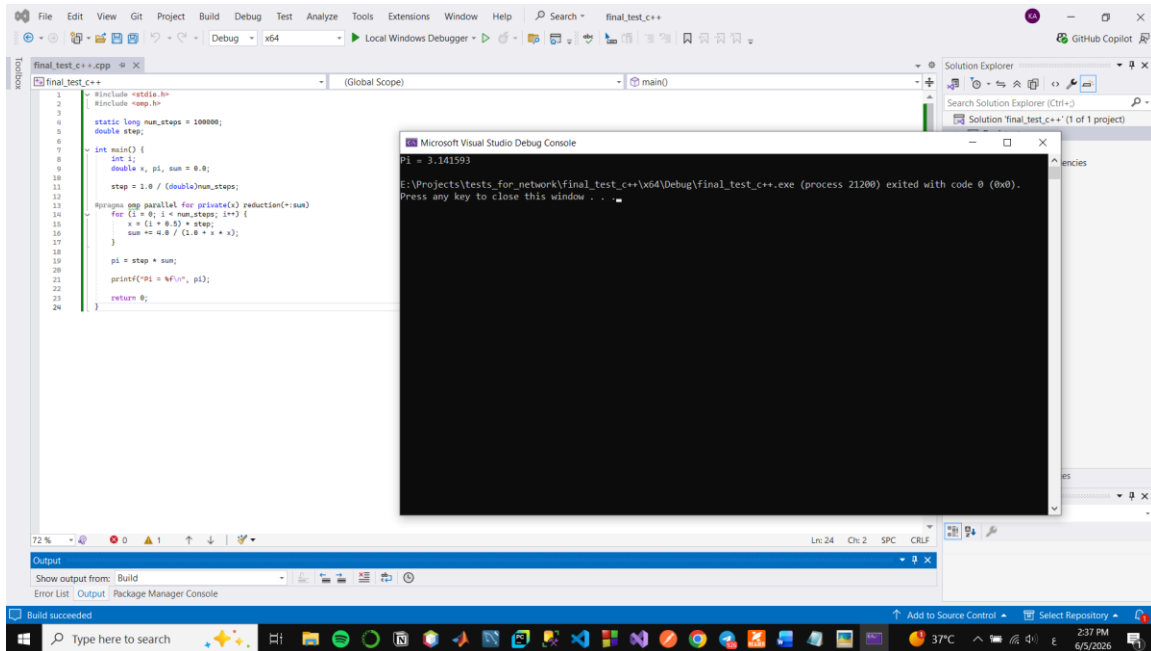
Source Code



```
1 #include <stdio.h>
2 #include <omp.h>
3
4 static long num_steps = 100000;
5 double step;
6
7 int main()
8 {
9     int i;
10    double x, pi, sum = 0.0;
11
12    step = 1.0 / (double)num_steps;
13
14    #pragma omp parallel for private(x) reduction(+ : sum)
15    for (i = 0; i < num_steps; i++)
16    {
17        x = (i + 0.5) * step;
18        sum += 4.0 / (1.0 + x * x);
19    }
20
21    pi = step * sum;
22
23    printf("Pi = %f\n", pi);
24
25    return 0;
26 }
```

Execution Results

Shorter, highly readable code with the best execution performance among all the previous variations.



Conclusion

The `reduction` clause is the standard, cleanest, and most optimized way to handle accumulative variables inside parallel loops in OpenMP.

Problem 9

Problem Information

Problem Title Histogram (Serial Version)

Lecture Number 3

Problem Description

Categorizing an array of values and distributing them into specific buckets based on their numerical range.

Parallelization Approach

None (Serial execution).

Solution Explanation

An array of values ``arr`` is initialized. A loop iterates through all elements, calling a ``sample`` function to determine the appropriate bucket, and increments the counter for that bucket in the ``hist`` array.

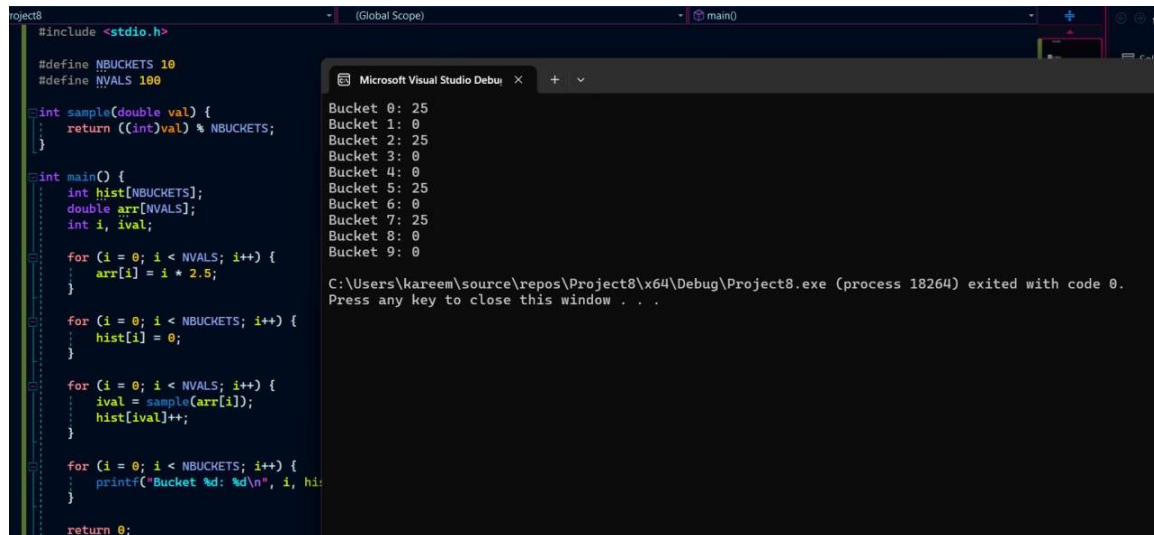
Source Code



```
1  #include <stdio.h>
2
3  #define NBUCKETS 10
4  #define NVALS 100
5
6  int sample(double val)
7  {
8      return ((int)val) % NBUCKETS;
9  }
10
11 int main()
12 {
13     int hist[NBUCKETS];
14     double arr[NVALS];
15     int i, ival;
16
17     for (i = 0; i < NVALS; i++)
18     {
19         arr[i] = i * 2.5;
20     }
21
22     for (i = 0; i < NBUCKETS; i++)
23     {
24         hist[i] = 0;
25     }
26
27     for (i = 0; i < NVALS; i++)
28     {
29         ival = sample(arr[i]);
30         hist[ival]++;
31     }
32
33     for (i = 0; i < NBUCKETS; i++)
34     {
35         printf("Bucket %d: %d\n", i, hist[i]);
36     }
37
38     return 0;
39 }
```

Execution Results

Correct distribution of the 100 values across 10 buckets.



```
#include <stdio.h>

#define NBUCKETS 10
#define NVALS 100

int sample(double val) {
    return ((int)val) % NBUCKETS;
}

int main() {
    int hist[NBUCKETS];
    double arr[NVALS];
    int i, ival;

    for (i = 0; i < NVALS; i++) {
        arr[i] = i * 2.5;
    }

    for (i = 0; i < NBUCKETS; i++) {
        hist[i] = 0;
    }

    for (i = 0; i < NVALS; i++) {
        ival = sample(arr[i]);
        hist[ival]++;
    }

    for (i = 0; i < NBUCKETS; i++) {
        printf("Bucket %d: %d\n", i, hist[i]);
    }

    return 0;
}
```

Microsoft Visual Studio Debug Console Output:

```
Bucket 0: 25
Bucket 1: 0
Bucket 2: 25
Bucket 3: 0
Bucket 4: 0
Bucket 5: 25
Bucket 6: 0
Bucket 7: 25
Bucket 8: 0
Bucket 9: 0

C:\Users\kareem\source\repos\Project8\x64\Debug\Project8.exe (process 18264) exited with code 0.
Press any key to close this window . . .
```

Conclusion

Stable performance, but the categorization process can become a severe bottleneck when dealing with millions of records.

Problem 10

Problem Information

Problem Title Histogram (Parallel with Simple Locks)

Lecture Number 3

Problem Description

Accelerating the categorization process using multiple threads while protecting the bucket data from race conditions during concurrent updates.

Parallelization Approach

Using OpenMP locks (`omp_lock_t`) to assign an independent lock to every single bucket.

Solution Explanation

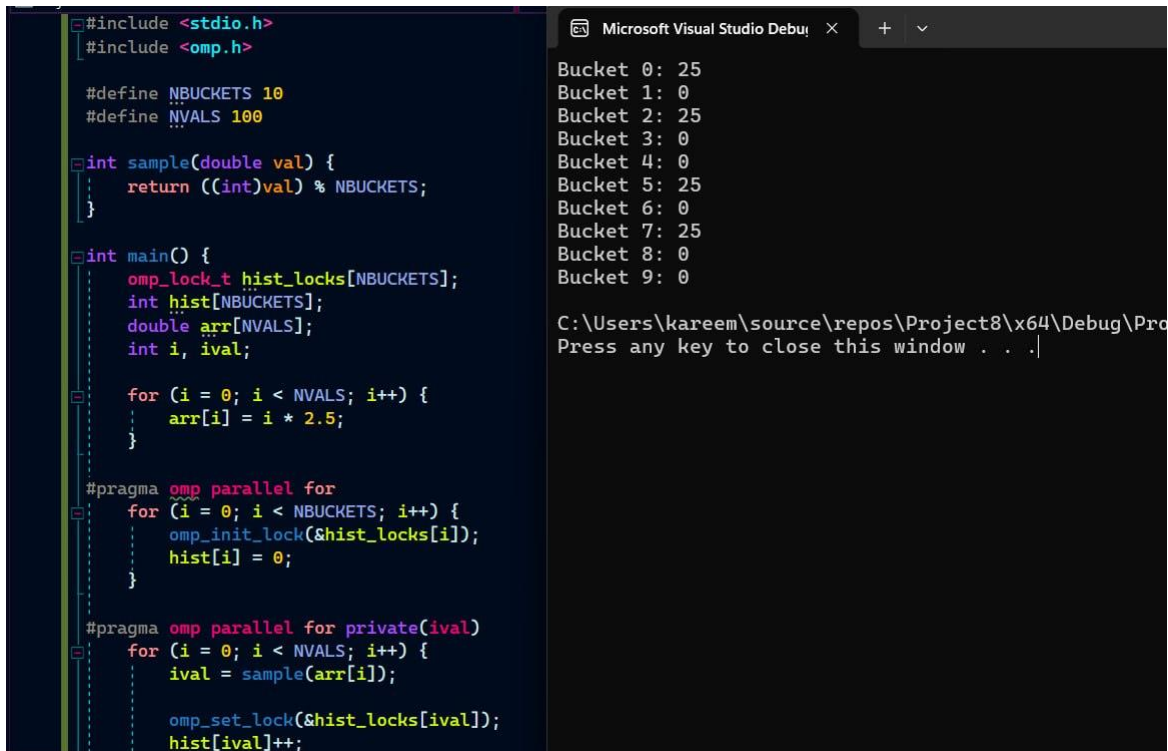
Locks are initialized first. Inside the `#pragma omp parallel for` loop, each thread determines the bucket for its assigned element, locks only that specific bucket, increments its value, and then unlocks it.

Source Code

```
1 #include <stdio.h>
2 #include <omp.h>
3
4 #define NBUCKETS 10
5 #define NVALS 100
6
7 int sample(double val)
8 {
9     return ((int)val) % NBUCKETS;
10 }
11
12 int main()
13 {
14     omp_lock_t hist_locks[NBUCKETS];
15     int hist[NBUCKETS];
16     double arr[NVALS];
17     int i, ival;
18
19     for (i = 0; i < NVALS; i++)
20     {
21         arr[i] = i * 2.5;
22     }
23
24 #pragma omp parallel for
25     for (i = 0; i < NBUCKETS; i++)
26     {
27         omp_init_lock(&hist_locks[i]);
28         hist[i] = 0;
29     }
30
31 #pragma omp parallel for private(ival)
32     for (i = 0; i < NVALS; i++)
33     {
34         ival = sample(arr[i]);
35         omp_set_lock(&hist_locks[ival]);
36         hist[ival]++;
37         omp_unset_lock(&hist_locks[ival]);
38     }
39
40     for (i = 0; i < NBUCKETS; i++)
41     {
42         printf("Bucket %d: %d\n", i, hist[i]);
43         omp_destroy_lock(&hist_locks[i]);
44     }
45
46     return 0;
47 }
```

Execution Results

Fast and accurate categorization. The fine-grained locking prevents unnecessary waiting times, as threads only block each other if they try to update the exact same bucket simultaneously.



The screenshot shows the Microsoft Visual Studio Debug Console. On the left, the C++ source code is displayed with syntax highlighting. The code defines 10 buckets and 100 values, then processes them in two parallel loops using OpenMP. The first loop initializes the buckets, and the second loop updates them based on a sample function. On the right, the output window shows the final state of the buckets: Bucket 0: 25, Bucket 1: 0, Bucket 2: 25, Bucket 3: 0, Bucket 4: 0, Bucket 5: 25, Bucket 6: 0, Bucket 7: 25, Bucket 8: 0, and Bucket 9: 0. Below the output, a message indicates the program has finished and prompts the user to press any key to close the window.

```
#include <stdio.h>
#include <omp.h>

#define NBUCKETS 10
#define NVALS 100

int sample(double val) {
    return ((int)val) % NBUCKETS;
}

int main() {
    omp_lock_t hist_locks[NBUCKETS];
    int hist[NBUCKETS];
    double arr[NVALS];
    int i, ival;

    for (i = 0; i < NVALS; i++) {
        arr[i] = i * 2.5;
    }

    #pragma omp parallel for
    for (i = 0; i < NBUCKETS; i++) {
        omp_init_lock(&hist_locks[i]);
        hist[i] = 0;
    }

    #pragma omp parallel for private(ival)
    for (i = 0; i < NVALS; i++) {
        ival = sample(arr[i]);

        omp_set_lock(&hist_locks[ival]);
        hist[ival]++;
    }
}
```

Microsoft Visual Studio Debug Console

Bucket 0: 25
Bucket 1: 0
Bucket 2: 25
Bucket 3: 0
Bucket 4: 0
Bucket 5: 25
Bucket 6: 0
Bucket 7: 25
Bucket 8: 0
Bucket 9: 0

C:\Users\kareem\source\repos\Project8\x64\Debug\Project8.exe
Press any key to close this window . . .

Conclusion

Arrays of locks offer an excellent solution for problems requiring sporadic updates to shared data structures without halting the entire parallel execution

Problem 11

Problem Information

Problem Title Runtime Library Routines

Lecture Number 3

Problem Description

Exploring how to use built-in OpenMP library functions to dynamically control and query the parallel environment.

Parallelization Approach

Using runtime C functions instead of pragmas for configuration, alongside `#pragma omp single` to restrict execution of a block to one thread.

Solution Explanation

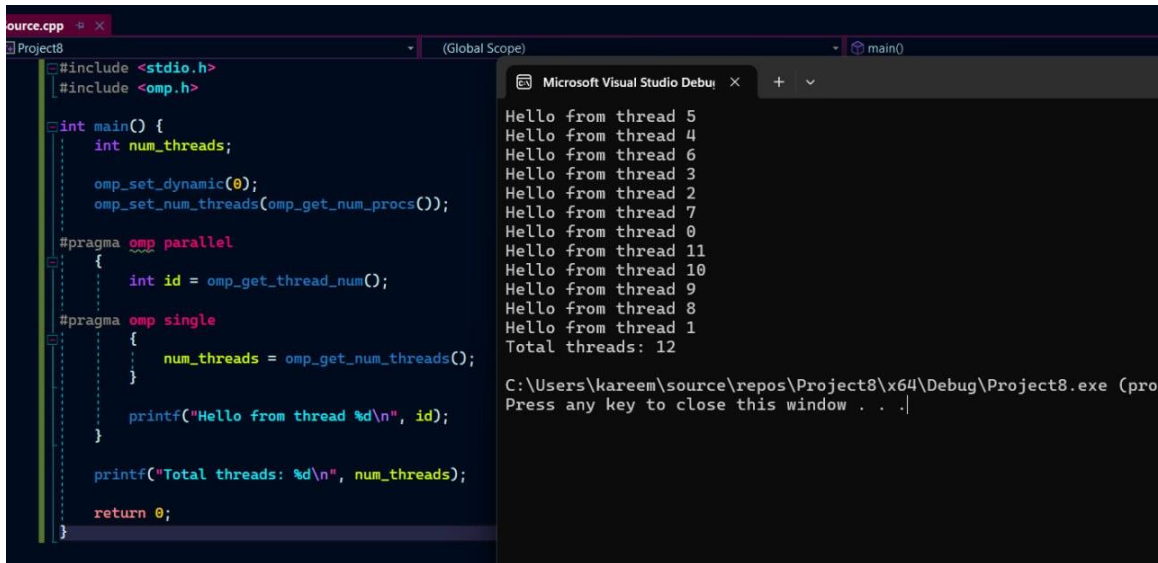
The code disables dynamic threading and sets the thread count to match the number of physical processors using `omp_get_num_procs()`. Inside the parallel region, each thread prints its ID, while a `single` block ensures the total thread count is queried only once.

Source Code

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int main()
5  {
6      int num_threads;
7
8      omp_set_dynamic(0);
9      omp_set_num_threads(omp_get_num_procs());
10
11  #pragma omp parallel
12  {
13      int id = omp_get_thread_num();
14
15  #pragma omp single
16  {
17      num_threads = omp_get_num_threads();
18  }
19
20      printf("Hello from thread %d\n", id);
21  }
22
23      printf("Total threads: %d\n", num_threads);
24
25      return 0;
26  }
```

Execution Results

Prints individual thread IDs followed by the total number of spawned threads.



```
source.cpp
Project8
(Global Scope)
main()

#include <stdio.h>
#include <omp.h>

int main() {
    int num_threads;

    omp_set_dynamic(0);
    omp_set_num_threads(omp_get_num_procs());

    #pragma omp parallel
    {
        int id = omp_get_thread_num();

        #pragma omp single
        {
            num_threads = omp_get_num_threads();
        }

        printf("Hello from thread %d\n", id);

        printf("Total threads: %d\n", num_threads);

        return 0;
    }
}

Microsoft Visual Studio Debug
Hello from thread 5
Hello from thread 4
Hello from thread 6
Hello from thread 3
Hello from thread 2
Hello from thread 7
Hello from thread 0
Hello from thread 11
Hello from thread 10
Hello from thread 9
Hello from thread 8
Hello from thread 1
Total threads: 12
C:\Users\kareem\source\repos\Project8\x64\Debug\Project8.exe (pro
Press any key to close this window . . .)
```

Conclusion

OpenMP runtime routines provide programmers with high flexibility to dynamically tune application performance based on the specific hardware running the program.

Problem 12

Problem Information

Problem Title Mandelbrot Area (Serial Version)

Lecture Number 3

Problem Description

Calculating the area of a Mandelbrot set by checking a 2D grid of complex points to see if they escape the set within a maximum number of iterations.

Parallelization Approach

None (Serial execution).

Solution Explanation

The program uses nested loops to traverse the grid. It calls ``testpoint`` for each coordinate to perform the complex arithmetic iterations. The global variable ``numoutside`` tracks escaped points.

Source Code

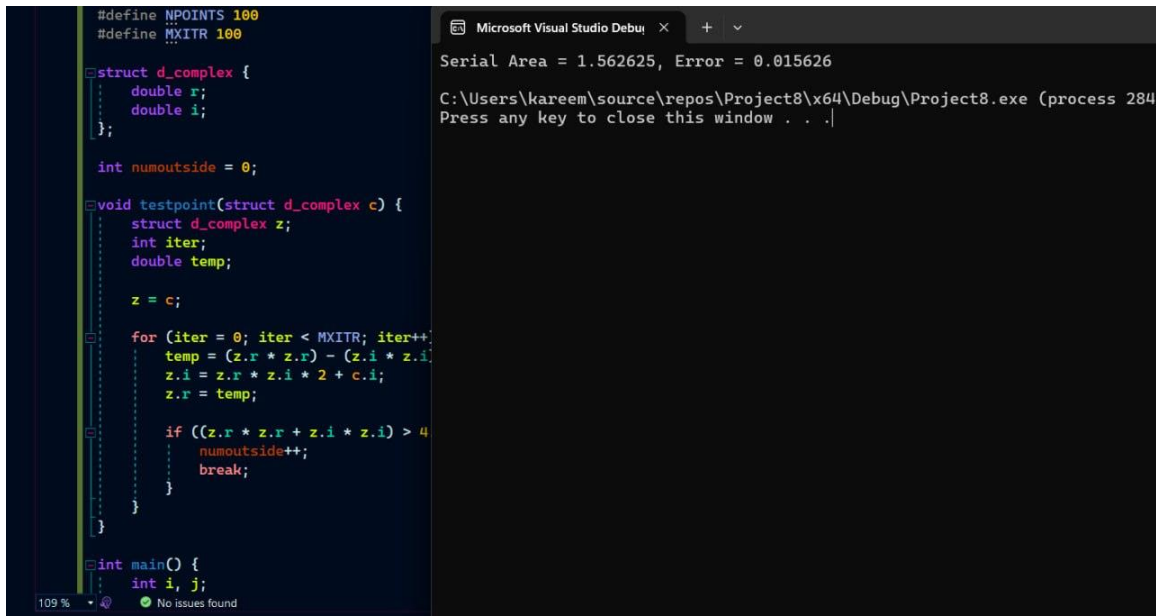
```

1  #include <stdio.h>
2
3  #define NPOINTS 100
4  #define MXITR 100
5
6  struct d_complex
7  {
8      double r;
9      double i;
10 };
11
12 int numoutside = 0;
13
14 void testpoint(struct d_complex c)
15 {
16     struct d_complex z;
17     int iter;
18     double temp;
19
20     z = c;
21
22     for (iter = 0; iter < MXITR; iter++)
23     {
24         temp = (z.r * z.r) - (z.i * z.i) + c.r;
25         z.i = z.r * z.i * 2 + c.i;
26         z.r = temp;
27
28         if ((z.r * z.r + z.i * z.i) > 4.0)
29         {
30             numoutside++;
31             break;
32         }
33     }
34 }
35
36 int main()
37 {
38     int i, j;
39     struct d_complex c;
40     double area, error, eps = 1.0e-5;
41
42     numoutside = 0;
43
44     for (i = 0; i < NPOINTS; i++)
45     {
46         for (j = 0; j < NPOINTS; j++)
47         {
48             c.r = -2.0 + 2.5 * (double)(i) / (double)(NPOINTS) + eps;
49             c.i = 1.125 * (double)(j) / (double)(NPOINTS) + eps;
50             testpoint(c);
51         }
52     }
53
54     area = 2.0 * 2.5 * 1.125 *
55           (double)(NPOINTS * NPOINTS - numoutside) /
56           (double)(NPOINTS * NPOINTS);
57
58     error = area / (double)NPOINTS;
59
60     printf("Serial Area = %f, Error = %f\n", area, error);
61
62     return 0;
63 }

```


Execution Results

Calculates the area accurately (based on a 100x100 grid).
Execution takes a noticeable amount of time due to the computationally dense nature of fractals.



The screenshot displays the Microsoft Visual Studio IDE. On the left, the C source code is visible, defining constants for NPOINTS (100) and MXITR (100), and implementing a testpoint function that iterates through a grid of points to calculate the area of the Mandelbrot set. The main function iterates over the grid. On the right, the Microsoft Visual Studio Debug Console shows the output: 'Serial Area = 1.562625, Error = 0.015626'. Below this, the file path 'C:\Users\kareem\source\repos\Project8\x64\Debug\Project8.exe (process 284)' and a prompt 'Press any key to close this window . . .|' are visible.

```
#define NPOINTS 100
#define MXITR 100

struct d_complex {
    double r;
    double i;
};

int numoutside = 0;

void testpoint(struct d_complex c) {
    struct d_complex z;
    int iter;
    double temp;

    z = c;

    for (iter = 0; iter < MXITR; iter++)
        temp = (z.r * z.r) - (z.i * z.i);
        z.i = z.r * z.i * 2 + c.i;
        z.r = temp;

        if ((z.r * z.r + z.i * z.i) > 4)
            numoutside++;
            break;
    }
}

int main() {
    int i, j;
}
```

Serial Area = 1.562625, Error = 0.015626
C:\Users\kareem\source\repos\Project8\x64\Debug\Project8.exe (process 284)
Press any key to close this window . . .|

Conclusion

This is a classic example of an embarrassingly parallel problem with heavy computational requirements, making it an ideal candidate for parallelization.

Problem 13

Problem Information

Problem Title Mandelbrot Area (Parallel Version)

Lecture Number 3

Problem Description

Parallelizing the nested loops to accelerate the Mandelbrot area calculation while protecting the shared accumulator variable.

Parallelization Approach

Using `#pragma omp parallel for` with specific data-sharing clauses `private`, `firstprivate`, and using `atomic` inside the test function.

Solution Explanation

The outer loop `i` is distributed among threads. `firstprivate(eps)` ensures each thread gets an initialized, independent copy of the epsilon value. Inside the `testpoint` function, the `#pragma omp atomic` directive protects the `numoutside` counter from race conditions.

Source Code

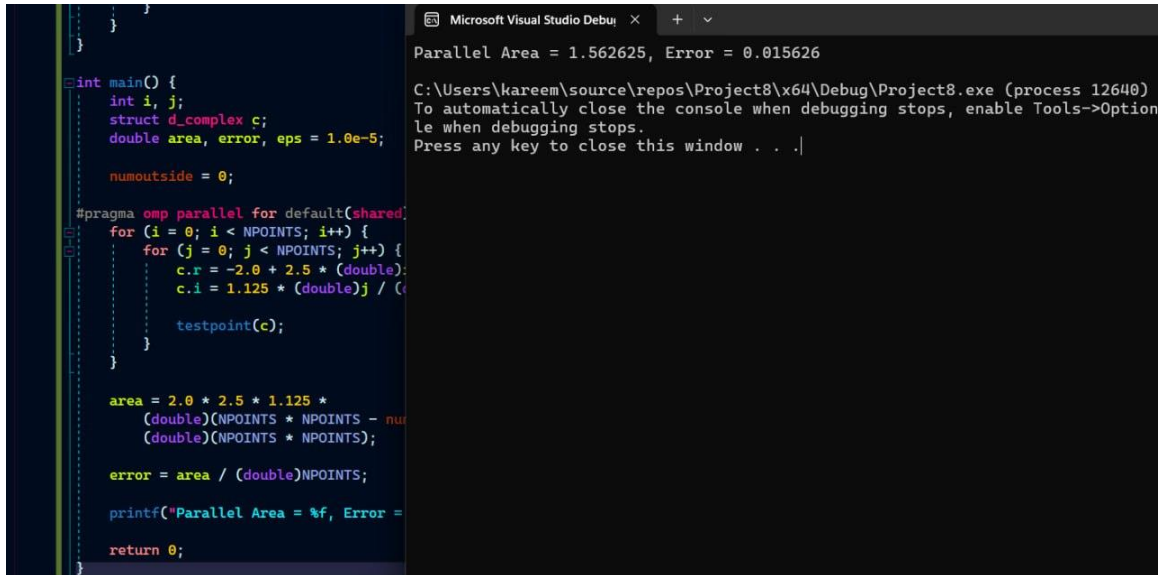
```

1  #include <stdio.h>
2  #include <omp.h>
3
4  #define NPOINTS 100
5  #define MXITR 100
6
7  struct d_complex
8  {
9      double r;
10     double i;
11 };
12
13 int numoutside = 0;
14
15 void testpoint(struct d_complex c)
16 {
17     struct d_complex z;
18     int iter;
19     double temp;
20
21     z = c;
22
23     for (iter = 0; iter < MXITR; iter++)
24     {
25         temp = (z.r * z.r) - (z.i * z.i) + c.r;
26         z.i = z.r * z.i * 2 + c.i;
27         z.r = temp;
28
29         if ((z.r * z.r + z.i * z.i) > 4.0)
30         {
31             #pragma omp atomic
32             numoutside++;
33             break;
34         }
35     }
36 }
37
38 int main()
39 {
40     int i, j;
41     struct d_complex c;
42     double area, error, eps = 1.0e-5;
43
44     numoutside = 0;
45
46     #pragma omp parallel for default(shared) private(c, j) firstprivate(eps)
47     for (i = 0; i < NPOINTS; i++)
48     {
49         for (j = 0; j < NPOINTS; j++)
50         {
51             c.r = -2.0 + 2.5 * (double)(i) / (double)(NPOINTS) + eps;
52             c.i = 1.125 * (double)(j) / (double)(NPOINTS) + eps;
53             testpoint(c);
54         }
55     }
56
57     area = 2.0 * 2.5 * 1.125 *
58           (double)(NPOINTS * NPOINTS - numoutside) /
59           (double)(NPOINTS * NPOINTS);
60
61     error = area / (double)NPOINTS;
62
63     printf("Parallel Area = %f, Error = %f\n", area, error);
64
65     return 0;
66 }

```

Execution Results

Massive improvement in processing speed due to the balanced distribution of intensive point-checking tasks across CPU cores.



The screenshot shows the Microsoft Visual Studio Debug console. On the left, the C++ source code is visible, featuring a nested loop with OpenMP parallelization. The code calculates a parallel area and error. On the right, the console output displays the results: 'Parallel Area = 1.562625, Error = 0.015626'. Below this, a message indicates the program path and provides instructions for closing the console window.

```
int main() {  
    int i, j;  
    struct d_complex c;  
    double area, error, eps = 1.0e-5;  
    numoutside = 0;  
  
    #pragma omp parallel for default(shared)  
    for (i = 0; i < NPOINTS; i++) {  
        for (j = 0; j < NPOINTS; j++) {  
            c.r = -2.0 + 2.5 * (double)i;  
            c.i = 1.125 * (double)j / (double)i;  
            testpoint(c);  
        }  
    }  
  
    area = 2.0 * 2.5 * 1.125 *  
        (double)(NPOINTS * NPOINTS - numoutside);  
    error = area / (double)NPOINTS;  
    printf("Parallel Area = %f, Error = %f\n", area, error);  
    return 0;  
}
```

Parallel Area = 1.562625, Error = 0.015626
C:\Users\kareem\source\repos\Project8\x64\Debug\Project8.exe (process 12640)
To automatically close the console when debugging stops, enable Tools->Options->Environment->Press any key to close this window . . .

Conclusion

Properly managing data-sharing attributes (scope variables) is crucial for both mathematical correctness and high performance in complex nested loops.

Problem 14

Problem Information

Problem Title Linked List Traversal (Serial Version)

Lecture Number 3

Problem Description

Traversing the nodes of a Linked List and processing each element's data individually.

Parallelization Approach

None (Serial execution).

Solution Explanation

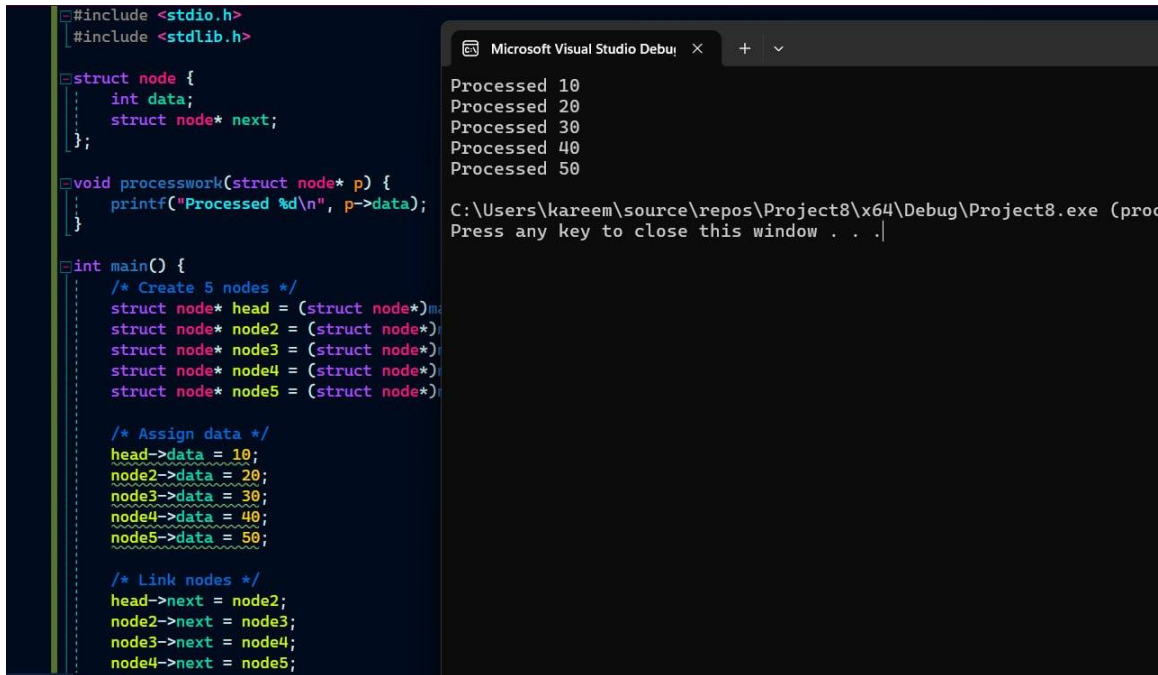
A pointer traverses the list using a `while` loop until it hits `NULL`, calling `processwork` sequentially on each node.

Source Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct node
5 {
6     int data;
7     struct node *next;
8 };
9
10 void processwork(struct node *p)
11 {
12     printf("Processed %d\n", p->data);
13 }
14
15 int main()
16 {
17     struct node *head = (struct node *)malloc(sizeof(struct node));
18
19     head->data = 10;
20     head->next = NULL;
21
22     struct node *p = head;
23
24     while (p != NULL)
25     {
26         processwork(p);
27         p = p->next;
28     }
29
30     free(head);
31
32     return 0;
33 }
```

Execution Results

Nodes are processed and printed sequentially in order.



The image shows a screenshot of a C program in Visual Studio. The program defines a linked list structure with five nodes, each containing an integer 'data' and a pointer to the next node. The main function creates five nodes with data values 10, 20, 30, 40, and 50, and links them sequentially. A function 'processwork' is called for each node, which prints the data value. The output window shows the following text:

```
Processed 10
Processed 20
Processed 30
Processed 40
Processed 50
```

The console path is C:\Users\kareem\source\repos\Project8\x64\Debug\Project8.exe (proj...

Conclusion

Linked Lists are inherently difficult to parallelize directly because accessing the next node strictly depends on traversing through the current node, preventing direct loop parallelization.

Problem 15

Problem Information

Problem Title Linked List Traversal (Parallel using Array)

Lecture Number 3

Problem Description

Overcoming the parallelization constraints of a Linked List by transforming its structure into an array to enable thread distribution.

Parallelization Approach

Two-phase execution First, sequentially harvest node pointers into an array. Second, parallelize the array traversal using `#pragma omp parallel for``.

Solution Explanation

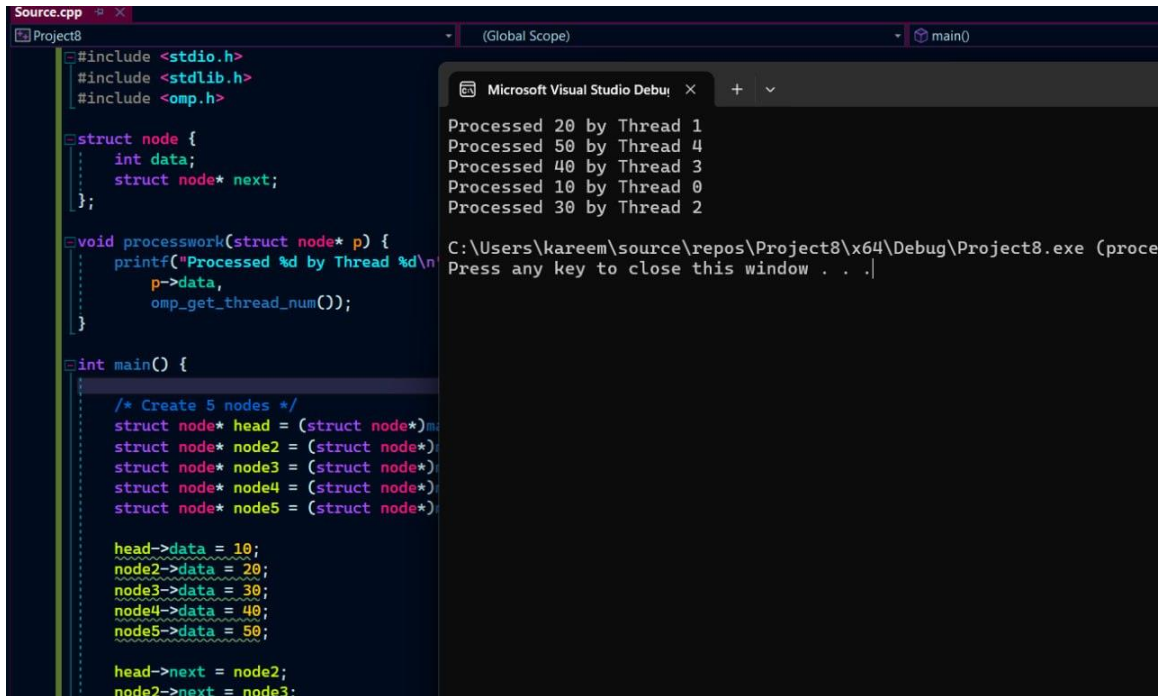
An initial serial `while`` loop walks the list and stores the memory address of each node into `parr``. Next, an OpenMP `for`` loop with `schedule(static, 1)`` distributes the pointers among threads for concurrent processing.

Source Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <omp.h>
4
5 struct node
6 {
7     int data;
8     struct node *next;
9 };
10
11 void processwork(struct node *p)
12 {
13     printf("Processed %d by Thread %d\n",
14           p->data,
15           omp_get_thread_num());
16 }
17
18 int main()
19 {
20     struct node *head = (struct node *)malloc(sizeof(struct node));
21
22     head->data = 10;
23     head->next = NULL;
24
25     struct node *p = head;
26     int count = 1;
27     struct node *parr[1];
28     int i;
29
30     for (i = 0; i < count; i++)
31     {
32         parr[i] = p;
33         p = p->next;
34     }
35
36     #pragma omp parallel
37     {
38         #pragma omp for schedule(static, 1)
39         for (i = 0; i < count; i++)
40         {
41             processwork(parr[i]);
42         }
43     }
44
45     free(head);
46
47     return 0;
48 }
```


Execution Results

Node data is processed concurrently by multiple threads.



The screenshot displays the Visual Studio IDE with a C++ source file named 'Source.cpp' and a debug console window. The source code defines a linked list structure and a function to process its nodes concurrently using OpenMP. The main function creates five nodes with data values 10, 20, 30, 40, and 50, and links them in sequence. The debug console shows the output of the program, indicating that each node's data was processed by a different thread.

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

struct node {
    int data;
    struct node* next;
};

void processwork(struct node* p) {
    printf("Processed %d by Thread %d\n", p->data, omp_get_thread_num());
}

int main() {
    /* Create 5 nodes */
    struct node* head = (struct node*)malloc(sizeof(struct node));
    struct node* node2 = (struct node*)malloc(sizeof(struct node));
    struct node* node3 = (struct node*)malloc(sizeof(struct node));
    struct node* node4 = (struct node*)malloc(sizeof(struct node));
    struct node* node5 = (struct node*)malloc(sizeof(struct node));

    head->data = 10;
    node2->data = 20;
    node3->data = 30;
    node4->data = 40;
    node5->data = 50;

    head->next = node2;
    node2->next = node3;
```

Processed 20 by Thread 1
Processed 50 by Thread 4
Processed 40 by Thread 3
Processed 10 by Thread 0
Processed 30 by Thread 2
C:\Users\kareem\source\repos\Project8\x64\Debug\Project8.exe (process)
Press any key to close this window . . .

Conclusion

This pointer harvesting trick is effective but requires allocating extra memory for the array and is impractical if the list size is massive or dynamically changing during traversal.

Problem 16

Problem Information

Problem Title Linked List Traversal (Parallel using C++ stdvector)

Lecture Number 3

Problem Description

Applying the array harvesting technique dynamically using standard C++ libraries.

Parallelization Approach

Collecting node pointers sequentially using a dynamic ``stdvector``, then processing the vector in parallel.

Solution Explanation

``stdvector`` eliminates the need to know the list size beforehand. After sequentially pushing all node pointers into ``nodelist``, a ``#pragma omp parallel for schedule(static,1)`` block processes the elements concurrently.

Source Code

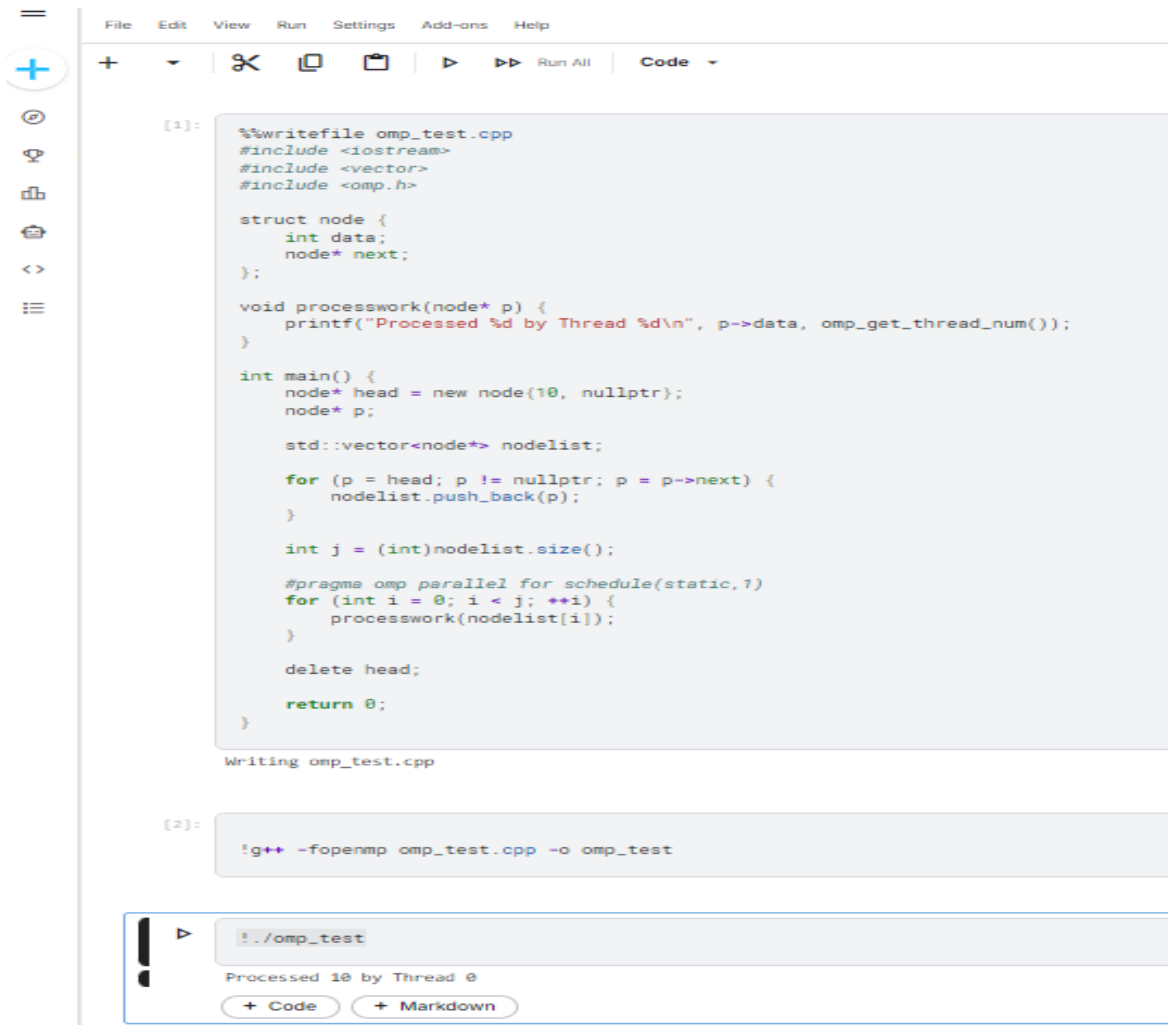
```

1  #include <iostream>
2  #include <vector>
3  #include <omp.h>
4
5          struct node
6  {
7      int data;
8      node *next;
9  };
10
11 void processwork(node *p)
12 {
13     printf("Processed %d by Thread %d\n", p->data, omp_get_thread_num());
14 }
15
16 int main()
17 {
18     node *head = new node{10, nullptr};
19     node *p;
20
21     std::vector<node *> nodelist;
22
23     for (p = head; p != nullptr; p = p->next)
24     {
25         nodelist.push_back(p);
26     }
27
28     int j = (int)nodelist.size();
29
30 #pragma omp parallel for schedule(static, 1)
31     for (int i = 0; i < j; ++i)
32     {
33         processwork(nodelist[i]);
34     }
35
36     delete head;
37
38     return 0;
39 }

```

Execution Results

Efficient concurrent processing of the nodes using available threads.



```
[1]: %writefile omp_test.cpp
#include <iostream>
#include <vector>
#include <omp.h>

struct node {
    int data;
    node* next;
};

void processwork(node* p) {
    printf("Processed %d by Thread %d\n", p->data, omp_get_thread_num());
}

int main() {
    node* head = new node{10, nullptr};
    node* p;

    std::vector<node*> nodelist;

    for (p = head; p != nullptr; p = p->next) {
        nodelist.push_back(p);
    }

    int j = (int)nodelist.size();

    #pragma omp parallel for schedule(static,1)
    for (int i = 0; i < j; ++i) {
        processwork(nodelist[i]);
    }

    delete head;

    return 0;
}

Writing omp_test.cpp

[2]: !g++ -fopenmp omp_test.cpp -o omp_test

!./omp_test
Processed 10 by Thread 0
+ Code + Markdown
```

Conclusion

Modern C++ features simplify parallel programming by handling dynamic memory management under the hood, making the code cleaner and less error-prone than raw C arrays.

Problem 17

Problem Information

Problem Title Fibonacci with Tasks (Serial Version)

Lecture Number 3

Problem Description

Computing a specific number in the Fibonacci sequence using a recursive algorithm.

Parallelization Approach

None (Serial execution).

Solution Explanation

The function calls itself recursively to sum the two previous values $\text{Fib}(n-1) + \text{Fib}(n-2)$. The execution tree expands depth-first on a single thread.

Source Code



```
1  #include <stdio.h>
2
3  int fib(int n)
4  {
5      int x, y;
6
7      if (n < 2)
8          return n;
9
10     x = fib(n - 1);
11     y = fib(n - 2);
12
13     return x + y;
14 }
15
16 int main()
17 {
18     int n = 15;
19     int result = fib(n);
20
21     printf("Serial Fib(%d) = %d\n", n, result);
22
23     return 0;
24 }
```

Execution Results

Accurately prints the result (610 for n=15).

The screenshot shows a Jupyter Notebook with the following content:

```
[4]: %%writefile omp_test.cpp
      #include <stdio.h>

      int fib(int n) {
          int x, y;

          if (n < 2) return n;

          x = fib(n - 1);
          y = fib(n - 2);

          return x + y;
      }

      int main() {
          int n = 15;
          int result = fib(n);

          printf("Serial Fib(%d) = %d\n", n, result);

          return 0;
      }
```

Overwriting omp_test.cpp

```
[5]: !g++ -fopenmp omp_test.cpp -o omp_test
```

Below the code cells, there is a terminal output area showing the command `!./omp_test` and its result: `Serial Fib(15) = 610`. At the bottom of the terminal area are two buttons: `+ Code` and `+ Markdown`.

Conclusion

The recursive Fibonacci solution has exponential time complexity $O(2^N)$, making it the perfect benchmark for demonstrating unstructured parallelism using Tasks.

Problem 18

Problem Information

Problem Title Fibonacci with Tasks (Parallel Version)

Lecture Number 3

Problem Description

Converting the deep, complex recursive tree into parallel tasks using OpenMP.

Parallelization Approach

Using `#pragma omp task` to spawn a distinct unit of work for each recursive call, and `#pragma omp taskwait` to synchronize them.

Solution Explanation

A `single` thread launches the initial recursive call. Inside the function, the calculations for `fib(n-1)` and `fib(n-2)` are wrapped in `#pragma omp task`. These tasks are thrown into a pool where idle threads pick them up. The `#pragma omp taskwait` ensures the parent thread doesn't add `x` and `y` until both sub-tasks finish.

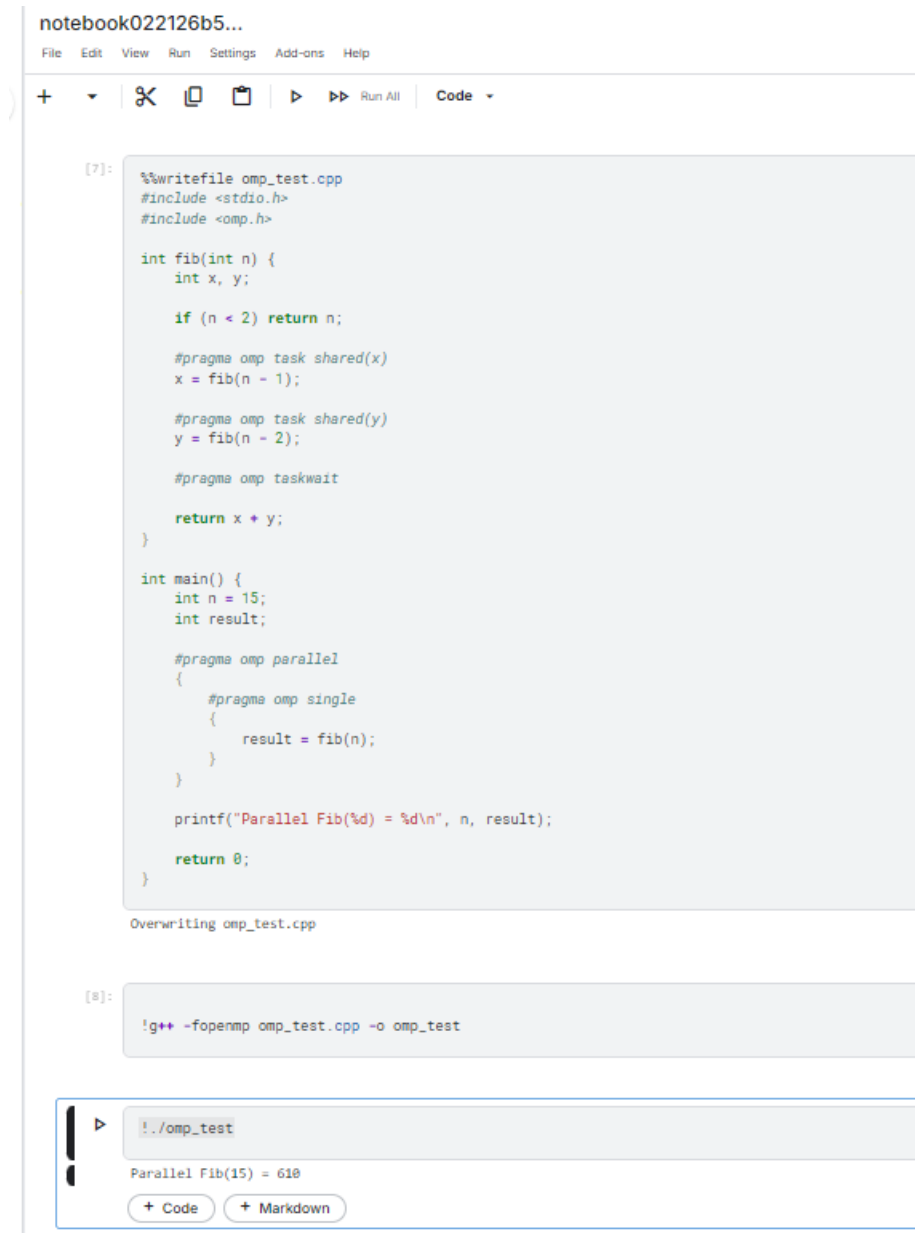
Source Code



```
1  #include <stdio.h>
2  #include <omp.h>
3
4  int fib(int n)
5  {
6      int x, y;
7
8      if (n < 2)
9          return n;
10
11 #pragma omp task shared(x)
12     x = fib(n - 1);
13
14 #pragma omp task shared(y)
15     y = fib(n - 2);
16
17 #pragma omp taskwait
18
19     return x + y;
20 }
21
22 int main()
23 {
24     int n = 15;
25     int result;
26
27 #pragma omp parallel
28     {
29 #pragma omp single
30         {
31             result = fib(n);
32         }
33     }
34
35     printf("Parallel Fib(%d) = %d\n", n, result);
36
37     return 0;
38 }
```

Execution Results

Mathematically accurate results with significantly better execution times, as independent branches of the recursion tree are computed simultaneously.



The screenshot shows a Jupyter Notebook interface with the title "notebook022126b5...". The menu bar includes "File", "Edit", "View", "Run", "Settings", "Add-ons", and "Help". Below the menu is a toolbar with icons for adding cells, undo, redo, copy, paste, and running code. The notebook contains three cells:

```
[7]: %%writefile omp_test.cpp
#include <stdio.h>
#include <omp.h>

int fib(int n) {
    int x, y;

    if (n < 2) return n;

    #pragma omp task shared(x)
    x = fib(n - 1);

    #pragma omp task shared(y)
    y = fib(n - 2);

    #pragma omp taskwait

    return x + y;
}

int main() {
    int n = 15;
    int result;

    #pragma omp parallel
    {
        #pragma omp single
        {
            result = fib(n);
        }
    }

    printf("Parallel Fib(%d) = %d\n", n, result);

    return 0;
}
```

Overwriting omp_test.cpp

```
[8]: !g++ -fopenmp omp_test.cpp -o omp_test
```

```
!./omp_test
Parallel Fib(15) = 610
```

At the bottom of the output cell, there are two buttons: "+ Code" and "+ Markdown".

Conclusion

The OpenMP Task model is explicitly designed for, and is the absolute best approach to handling, recursive algorithms and dynamic loop conditions.

Problem 19

Problem Information

Problem Title Linked List with Tasks (Parallel Version)

Lecture Number 3

Problem Description

Solving the Linked List parallelization dilemma natively, without needing to copy pointers into an array, by utilizing dynamic tasks.

Parallelization Approach

Sequentially traversing the list with one thread (`#pragma omp single``), but generating a parallel task (`#pragma omp task``) for each node's processing workload.

Solution Explanation

A single thread walks through the linked list. Instead of doing the work, it acts as a dispatcher at every node, it creates a Task containing the node's pointer (via ``firstprivate(e)``) and throws it into the thread pool. The other threads in the parallel team immediately pick up these tasks and process them.

Source Code

```

1
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <omp.h>
5
6 typedef struct Element
7 {
8     int data;
9     struct Element *next;
10 } Element;
11
12 typedef struct List
13 {
14     Element *first;
15 } List;
16
17 void process(Element *e)
18 {
19     printf("Processing node with data: %d by Thread %d\n",
20           e->data,
21           omp_get_thread_num());
22 }
23
24 int main()
25 {
26     List *ml = (List *)malloc(sizeof(List));
27     Element *head = (Element *)malloc(sizeof(Element));
28
29     head->data = 100;
30     head->next = NULL;
31     ml->first = head;
32
33     Element *e;
34
35     #pragma omp parallel
36     {
37         #pragma omp single
38         {
39             for (e = ml->first; e; e = e->next)
40             {
41                 #pragma omp task firstprivate(e)
42                 {
43                     process(e);
44                 }
45             }
46         }
47     }
48
49     free(head);
50     free(ml);
51
52     return 0;
53 }

```

Execution Results

Concurrent processing of linked list nodes without sequential bottlenecks or memory overhead.

```
Overwriting omp_test.cpp
```

```
[11]: !g++ -fopenmp omp_test.cpp -o omp_test
```

```
!./omp_test
```

Processing node with data: 100 by Thread 2

+ Code + Markdown

Conclusion

Using Tasks is the most elegant, scalable, and professional way to parallelize dynamic data structures like Linked Lists, Trees, and Graphs in OpenMP.

Problem 20

Problem Information

MPI Hello World

Lecture Number 3

Problem Description

A fundamental introductory program for distributed memory parallelization using MPI (Message Passing Interface). The objective is to initialize the MPI environment and have each spawned process print its unique rank along with the total number of processes

Parallelization Approach

Distributed memory model using the MPI standard. Unlike OpenMP (which shares memory), each MPI process runs in its own completely isolated memory space and requires explicit initialization to communicate.

Solution Explanation

The program begins by calling MPI_Init() to set up the execution environment.

It retrieves the total number of processes in the default communicator using MPI_Comm_size(). It retrieves the unique ID (rank) of the current executing process using MPI_Comm_rank().

Every process independently executes the printf statement using its local copy of the rank and size variables.

Finally, MPI_Finalize() is called to safely tear down the MPI environment before the program exits.

Source Code

```
#include <mpi.h>

int main(int argc, char *argv[])
{
    int rank, size;

    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    printf("Hello world from process %d of %d processes.\n", rank, size);

    MPI_Finalize();

    return 0;
}
```

Execution Results

The order of the printed lines is entirely non-deterministic depending on which process reaches the printf statement first.

```
[1]: %writefile hello.c
      #include <stdio.h>
      #include <mpi.h>

      int main(int argc, char *argv[])
      {
          int rank, size;

          MPI_Init(&argc, &argv);

          MPI_Comm_size(MPI_COMM_WORLD, &size);
          MPI_Comm_rank(MPI_COMM_WORLD, &rank);

          printf("Hello world from process %d of %d processes.\n", rank, size);

          MPI_Finalize();

          return 0;
      }
```

Writing hello.c

```
[2]: !mpicc hello.c -o hello
```

```
▶ !mpiexec --allow-run-as-root -n 2 ./hello
```

```
Hello world from process 1 of 2 processes.
Hello world from process 0 of 2 processes.
```

+ Code

+ Markdown

Conclusion

This code serves as a vital sanity check to verify that the MPI compiler (mpicc) and the runtime execution environment (mpirun/mpiexec) are correctly configured to spawn and manage distributed processes.

Problem 21

Problem Information

MPI Ping-Pong (Point-to-Point Communication)

Lecture Number 3

Problem Description

A standard benchmark program designed to test network latency and communication overhead. The objective is to bounce a dynamically sized payload back and forth (like a ping-pong ball) between exactly two distinct MPI processes.

Parallelization Approach

Synchronous, point-to-point communication using blocking MPI routines (MPI_Send and MPI_Recv).

Solution Explanation

Validation: The program first checks if exactly 2 processes (`numprocs == 2`) are running. If not, Process 0 prints an error message, and the program terminates safely.

Dynamic Payload: It enters a while loop where the message size (`msg_size`) starts at 64 bytes and doubles exponentially (`64 \to 128 \to 256 \dots`) until it reaches roughly 10 megabytes. Inside the loop, a buffer is dynamically allocated using `malloc`.

Process 0 (The Initiator): If the rank is 0, the process uses `MPI_Send` to transmit the buffer to Process 1. It then immediately halts and waits using `MPI_Recv` to get the buffer back from Process 1.

Process 1 (The Reflector): If the rank is 1, the process first waits to catch the buffer using `MPI_Recv`, and then immediately throws it back to Process 0 using `MPI_Send`.

Memory Management: The dynamically allocated buffer is freed at the end of each loop iteration to prevent memory leaks.

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char *argv[])
{
    int numprocs, rank;
    int tag = 100;
    int msg_size = 64;

    char *buf;
    MPI_Status status;

    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (numprocs != 2)
    {
        if (rank == 0)
            printf("The number of processes must be two!\n");

        MPI_Finalize();
        return 0;
    }

    printf("MPI process %d started...\n", rank);

    while (msg_size < 100000000)
    {
        msg_size *= 2;

        buf = (char *)malloc(msg_size);

        if (rank == 0)
        {
            MPI_Send(buf, msg_size, MPI_BYTE, 1, tag, MPI_COMM_WORLD);

            printf("Message of length %d sent to process 1\n", msg_size);

            MPI_Recv(buf, msg_size, MPI_BYTE, 1, tag,
                    MPI_COMM_WORLD, &status);
        }
        else
        {
            MPI_Recv(buf, msg_size, MPI_BYTE, 0, tag,
                    MPI_COMM_WORLD, &status);

            MPI_Send(buf, msg_size, MPI_BYTE, 0, tag,
                    MPI_COMM_WORLD);

            printf("Message of length %d sent to process 0\n", msg_size);
        }

        free(buf);
    }

    MPI_Finalize();

    return 0;
}
```

Execution Results

The console will display a sequence of messages from both processes confirming the transmission of data at exponentially increasing sizes (e.g., "Message of length 128 sent...", "Message of length 256 sent...").

```
!mpiexec --allow-run-as-root -n 2 ./sendrecv
```

```
MPI process 0 started...
Message of length 128 sent to process 1
MPI process 1 started...
Message of length 128 sent to process 0
Message of length 256 sent to process 0
Message of length 512 sent to process 0
Message of length 1024 sent to process 0
Message of length 2048 sent to process 0
Message of length 256 sent to process 1
Message of length 512 sent to process 1
Message of length 1024 sent to process 1
Message of length 2048 sent to process 1
Message of length 4096 sent to process 1
Message of length 4096 sent to process 0
Message of length 8192 sent to process 1
Message of length 8192 sent to process 0
Message of length 16384 sent to process 1
Message of length 32768 sent to process 1
Message of length 65536 sent to process 1
Message of length 16384 sent to process 0
Message of length 32768 sent to process 0
Message of length 65536 sent to process 0
Message of length 131072 sent to process 1
Message of length 131072 sent to process 0
Message of length 262144 sent to process 1
Message of length 262144 sent to process 0
Message of length 524288 sent to process 1
Message of length 524288 sent to process 0
Message of length 1048576 sent to process 1
Message of length 1048576 sent to process 0
Message of length 2097152 sent to process 1
Message of length 2097152 sent to process 0
Message of length 4194304 sent to process 1
Message of length 4194304 sent to process 0
Message of length 8388608 sent to process 1
Message of length 8388608 sent to process 0
Message of length 16777216 sent to process 1
Message of length 16777216 sent to process 0
```

Conclusion

The Ping-Pong topology is a classic method used in High-Performance Computing (HPC) to measure the bandwidth capabilities of a cluster. It perfectly demonstrates how blocking communication forces processes to synchronize their states in a distributed memory architecture.